

一、 写出下列程序段的运行结果（40 分）

1、（4 分）

```
char m;  
int a=3;  
m='B'+32; printf("%c",m);  
printf ("%d,%d", a==3, a = 3);  
m=0x123040; printf("%d",m);
```

2、（4 分）

```
int i=1, j=2, k=4;  
printf("%f", i-(float)(2/k));  
printf("%d", j>3<k);  
k+=(i+k)/10+0.5;  
printf("%d",k);  
printf("%d", (j=1)?3:0);
```

3、（4 分）

```
int a=5,y=0;  
do  
{  
a+=5;  
y+=a;  
printf("%d,%d", a, y);  
if (y>18) break;  
} while(a=5);
```

4、（4 分）

```
int  
p[7]={11,13,15,17,18,20,23},  
i=0,k=0;  
for (;p[i]%2;)  
{  
switch(p[i]%5)  
{  
case 1: k+=p[i]; break;  
case 3: k+=3;  
default: i++; continue;  
}  
i++;  
}  
printf("%d,%d",i,k);
```

5、（4 分）

```
char s[]="3210", *p;  
for (p=s+2; p>=s; p--)  
{  
printf("%s", p);  
}
```

6、（4 分）

```
int f()  
{  
static int j=0; int k=0, t=0;  
t += j;  
t += (j++)&&(++k);  
return t;  
}  
int main()  
{  
int i;  
for (i=0; i<3; i++)  
printf("%d", f());  
return 0;  
}
```

7、（4 分）

```
void fun(int i)  
{  
if (i)  
{  
fun(i/2);  
printf("%d",i);  
}  
}  
int main()  
{  
fun(9);  
return 0;  
}
```

8、（4 分）

union

```

{
    int a[3];
    struct { int a,b,c; } s;
}u={1,2,3};
int main()
{
    u.s.a=0; u.s.b=1; u.s.c=2;
    printf("%d%d", u.a[0],
u.b[2]);
    return 0;
}

```

0 2

9、(4分)

```
int s[][4]= {1,2,3,4,5,6,7,8};
```

1 2 3 4
5 6 7 8

```

int sum;
sum = *(s[0]+2) + *(s[1]+1);
printf("%d", sum);

```

10

10、(4分)

```

FILE *fp; int d;
fp=fopen("pi.tmp","w");
fputs("3.1415926", fp);
fclose(fp);
fp=fopen("pi.tmp","r");
fseek(fp, 4L, SEEK_SET);
fscanf(fp,"%d",&d);
printf("%ld",d);

```

二、改错题 (20 分)

以下程序实现：从键盘输入 20 个销售员的业绩数据，将这些数据存到磁盘文件上。程序中存在若干错误，指出错误所在位置并改正。

```

#define SIZE 20 /*第 1 行*/
#include <stdio.h> /*第 2 行*/
struct SELLER /*第 3 行*/
{ /*第 4 行*/
    char name[15]; /*第 5 行*/
    double sell; /*第 6 行*/
}; /*第 7 行*/
int save(struct SELLER sells[SIZE]) /*第 8 行*/
{ /*第 9 行*/
    FILE *fp; /*第 10 行*/
    int i; /*第 11 行*/
    if((fp=fopen("stulist","wb"))==NULL) /*第 12 行*/
    { /*第 13 行*/
        printf("cannot open file\n"); /*第 14 行*/
        return; /*第 15 行*/
    } /*第 16 行*/
    for(i=0;i<=SIZE;i++) /*第 17 行*/
    if(fwrite(&sells[i],sizeof(struct SELLER),1,fp)!=1) /*第 18 行*/
        printf("file write error\n"); /*第 19 行*/
    } /*第 20 行*/
} /*第 21 行*/
Int main() /*第 22 行*/
{ /*第 23 行*/
    Struct SELLER sells[SIZE]; /*第 24 行*/
    int i; /*第 25 行*/
    for(i=0;i<SIZE;i++); /*第 26 行*/
    { /*第 27 行*/
        scanf("%s%lf", sells[i].name, sells[i].sell);
    }
}

```

```

}
save(struct sells);
return 0;
}
/*第 28 行*/
/*第 29 行*/
/*第 30 行*/
/*第 31 行*/

```

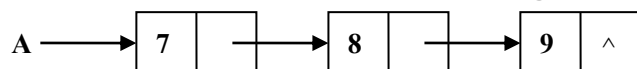
三、编程题

1、海滩上有一堆桃子，五只猴子来分。第一只猴子把这堆桃子凭据分为五份，多了一个，这只猴子把多的一个扔入海中，拿走了一份。第二只猴子把剩下的桃子又平均分成五份，又多了-一个，它同样把多的一个扔入海中，拿走了一份，第三、第四、第五只猴子都是这样做的，问海滩上原来最少有多少个桃子？（12分）

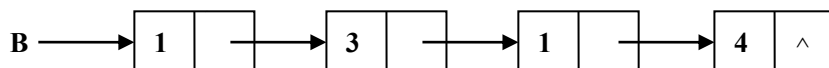
2、输入一个字符串，请编写一个函数统计连续相同字符及其数量。例如，输入“AAABBBCCCab”，返回“A3 B3 C3 a1 b1”及5；又如，输入“abCCbbbaC”，返回“a1 b1 C2 b3 a1 C1”和6。

要求：所编写的函数中应采用指针法来引用输入字符串中的元素。（14分）

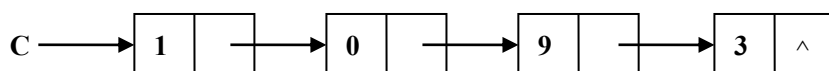
3、用链表存储一个正整数（类型unsigned long），如789被存储到链表A：



编写函数SimpleAdd(A, B)，参数A和B分别指向两个链表（表示两个数），函数返回一个新链表C，表示Simple A+B之和（加法没有进位，且输出的数的首位不能为0。类似 $125 + 932 = 57$ ， $5555 + 5555$ 的情况只有一个0）。例如当B为1314时，



执行C=SimpleAdd(A, B)，结果如下：



要求：（1）给出链表结点定义（2分）；

（2）编写函数SimpleAdd，实现题目要求的功能。（12分）

参考答案:

第一题:

题号	参考答案
1	b,1,3,64
2	1.000000,1,4,3
3	10,10,10,20,
4	4,14
5	102103210
6	023
7	1249
8	02
9	9
10	15926

第二题:

- 1、第 1 行, **#define SIZE 20;** 去掉;
- 2、第 7 行, 加;
- 3、第 8 行, int 改为 void
- 4、第 10 行, *fp
- 5、第 12 行, 第一个 == 改 =
- 6、第 17 行, 去掉 =
- 7、第 19 行后增加 fclose (fp)
- 8、第 23 行 **Struct** 改为 **struct**
- 9、第 25 行 去掉;
- 10、第 27 行, 加上&
- 11、第 29 行, 去掉 struct

第三题

1.

设第5个猴子拿完后 还剩下x个桃子 ($x \in \mathbb{N}$)

那么第4个猴子拿完后, 第五个猴子拿之前, 应有 $x*5/4+1$ 个桃子,

不妨设 $y=x*5/4+1$, 同理 第3个猴子拿完后, 第四个猴子拿之前, 应有

$y*5/4+1$ 个桃子

.....

也就是 每次把岛上剩余的桃子数乘5/4再加1 就是上一个猴子拿之前的桃子数,

设岛上最初有z个桃子 由于这是实际问题 则 $z \in \mathbb{N}$ 现在 我们只需要让x从1开始向上分别取 1, 2, 3, 4.....看哪个x能满足最后算出的z是整数即可

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
double x = 1.0, y, z;
```

```

int i;

for (x = 1.0;; x += 1.0)
{
    y = x;

    for (i = 0; i < 5; i++)
    {
        z = y * 5 / 4 + 1;

        y = z;
    }

    if ((int)z == z)
        break;
}

printf("%.0lf", z);

return 0;
}

```

最终结果是 3121（能摘这么多桃子 猴子也是蛮拼的）

2. 本题较简单，直接根据题意写代码即可

```

#include<stdio.h>

#include<string.h>

int main()
{
    char str[1001];

    int i, j, k, l, n = 0;

```

```

    gets(str);

    l = strlen(str);

    for (i = 0; i < l; i++)
    {
        for (k = 1, j = i + 1; j < l; j++)
        {
            if (str[j] == str[i]) //如果字符相同 那么k+1

                k++;

            else //如果不相同 统计结束

                break;
        }

        printf("%c%d ", str[i], k); //边统计边输出

        n++; //输出一次 紧跟着n自增1

        i = j - 1;
    }

    printf("%d", n); //最后输出n

    return 0;
}

```

3. 本题较难，实则链表版的不进位a+b

```

#include<stdio.h>

#include<stdlib.h>

#define len sizeof(struct num)

```

```

struct num
{
    int i;

    struct num *next;

    struct num *before;
};

struct num *creat(unsigned long n)
{
    struct num *p1, *p2;

    p1 = p2 = (struct num *)malloc(len);

    p1->i = n % 10;

    p1->next = NULL; //注意这里的p1是将来的尾节点 不是一般我们建立链表时的头节点 所以p1的next是NULL

    n = n / 10;

    while (n)
    {
        p1 = (struct num *)malloc(len);

        p1->i = n % 10;

        p1->next = p2; //注意这里是 p1->next=p2 不是一般情况的
        p2->next=p1 理由同上

        p2->before = p1;

        p2 = p1;

        n = n / 10;
    }
}

```

```

    }

    p2->before = NULL;

    return p2;
}

struct num *SimpleAdd(struct num *p, struct num *q)
{
    struct num *p1, *p2;

    while (p->next != NULL)

        p = p->next;

    while (q->next != NULL)

        q = q->next;    //先让p q指针都指向那两个要加的数字的个位 既然是
运算 肯定要从个位开始相加

    p2 = p1 = (struct num *)malloc(len);

    p1->i = (p->i + q->i) % 10;

    p1->next = NULL;

    p = p->before; q = q->before;

    while (q&&p)    //循环终止条件是 p q链表有一个是空就停止，链表为空表示
这个链表储存的数字已经加完了 但是另一个数字可能没加完 比如 1314+520
520先加完 但是1314还有千位的1没有加 我们得把它补上

    {

        p1 = (struct num *)malloc(len);

        p1->i = (p->i + q->i) % 10;

        p1->next = p2;

```



```

    p2->before = p1;

    p2 = p1;

    p = p->before; q = q->before;
}

```

if (q == *NULL* && p != *NULL*) //下面的语句就是为了补上没有加的位数 补起来很简单 比如1314+520 我们已经算了 0+4 2+1 5+3 只要让5+3=8所在的节点的before指向1314的千位1即可，别忘了让1314链表的千位的next指向8也就是p2

```

{

    p2->before = p;

    p->next = p2;
}

if (q != NULL && p == NULL)
{

    p2->before = q;

    q->next = p2;
}

if (p == NULL && q == NULL)

    p2->before = NULL;

while (p2->before)

    p2 = p2->before;

while (p2->i == 0 && p2->next != NULL)
{

    p1 = p2;
}

```

```
p2 = p2->next;
```

```
p2->before = NULL;
```

```
free(p1);
```

```
} //这个语句就是为了消除如555+555多余存在的0，同时保证不把所有的0都
```

去掉 如555+555最后要保留一个0的

```
return p2;
```

```
}
```

```
int main()
```

```
{
```

```
unsigned long n, m;
```

```
struct num *p, *q, *sum;
```

```
scanf("%ld %ld", &n, &m);
```

```
p = creat(n); q = creat(m);
```

```
sum = SimpleAdd(p, q);
```

```
while (sum)
```

```
{
```

```
printf("%d", sum->i);
```

```
sum = sum->next;
```

```
}
```

```
return 0;
```

```
}
```